

FlexSim Point Cloud Importer User Guide

Simulation Bureau LLC · FlexScript utility · MIT License

Import an XYZ text point cloud into a FlexSim model as a colored point mesh. The importer creates a Visual Tool object, installs its On Draw trigger, and stores the mesh on the object. It also organizes source files beside the saved model.

This guide explains how to use the ImportXYZPointCloud.fsl library to import and display point clouds in FlexSim. Library workflow sections are provided for completion with instructions and screenshots. Standalone instructions for xyzPointCloudImport.fs and developer reference material appear in the appendices.

The library is expected to be compatible with FlexSim 2024 and later versions; however, it has not been tested with every FlexSim release from 2024 onward.

Important: Save your FlexSim model before importing. The importer uses the saved model location to create the PointCloudFiles folder. If the model has not been saved, the importer displays a message and stops; it does not open Save As automatically.

Features

- Select an .xyz file through a file browser.
- Read space-, comma-, or tab-delimited coordinates.
- Detect an optional single header row and optional RGB columns.
- Convert source coordinates into the model's length units.
- Create numbered PointCloud_N objects and corresponding folders.
- Optionally copy the source file beside the model.
- Adjust point size and cloud visibility through object labels.
- Report point count, skipped lines, and coordinate bounds.

The result is a visual point cloud. The importer does not create a surface, solid, collision geometry, or simulation routing network.

Requirements

- FlexSim with the ImportXYZPointCloud.fsl library installed.
- Windows, because the importer uses Windows paths and cmd.exe for source-file copying.
- A model saved to a writable folder.
- An .xyz text file and sufficient available memory to hold its points.

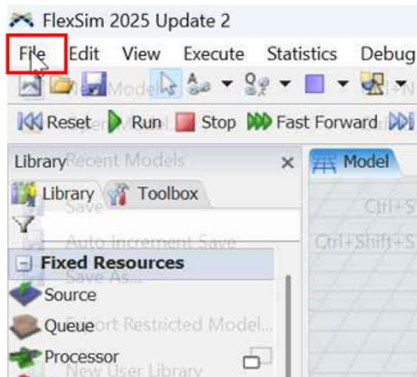
No separate Python installation or external conversion program is used by the importer.

Installing the Library

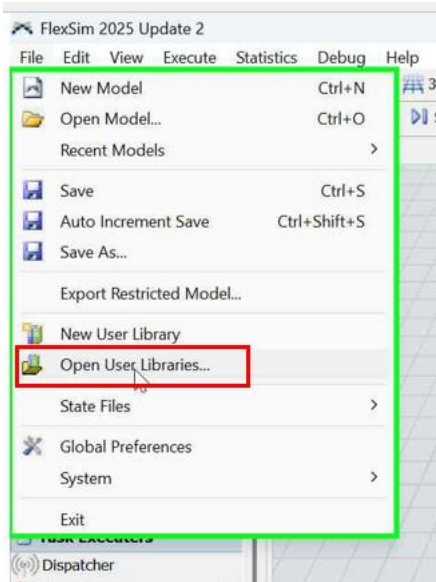
1. **Download the library from GitHub.**
Open the <https://github.com/SimulationBureau/FSPCImporter> and download ImportXYZPointCloud.fsl. The .fs source file is optional for users installing the library.
2. **Save the library to your computer.**
Save the file in a location where you can easily find it. If downloaded as part of a ZIP archive, extract the files before continuing.
3. **Open FlexSim.**
Launch FlexSim and open a new or existing model.

4. Load the library.

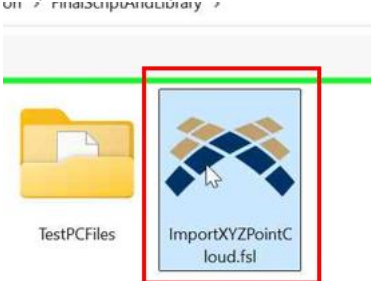
Step 1: User left click on "File (menu item)" in "FlexSim 2025 Update 2"



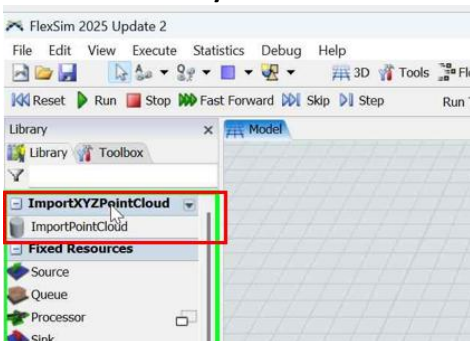
Step 2: User left click on "Open User Libraries... (menu item)"



Step 3: User left double click on "ImportXYZPointCloud.fsl (list item)" in "Select FlexSim Library files"



5. Confirm the library is loaded.



Preparing the Point Cloud

Point Cloud Test Files

The point cloud files used for testing and validation of this library are provided in .xyz format. These test files were created using an iPhone 17 Pro Max and the **PointFoundry** application, obtained through the Apple App Store.

Simulation Bureau LLC is not affiliated with, sponsored by, or endorsed by PointFoundry. PointFoundry was used only as one available method for generating point cloud data, and many other applications and scanning solutions may be used to create compatible .xyz point cloud files.

Point Cloud File Conversion

If a point cloud is not already available in .xyz format, it may be necessary to convert it before importing it into FlexSim. A number of point cloud processing applications are available for converting between common file formats.

One example is MeshLab, a widely used open-source application that supports importing and exporting a variety of 3D mesh and point cloud file types. Other conversion tools may also be used, provided that the resulting file is exported in a format compatible with this importer.

When converting or exporting a point cloud:

- Export the point cloud as an .xyz text file.
- Use Z-Up where the source application provides an axis-orientation option.
- Preserve RGB color information if color is required.
- Verify the exported column order and delimiter before importing.

Some conversion applications may export additional attributes such as intensity, normals, alpha values, or other point data. These additional columns can affect how the file is interpreted by the importer.

For exact column layout, supported delimiters, RGB formatting, header behavior, and parser limitations, see Supported Input Format.

Checking Units and Orientation

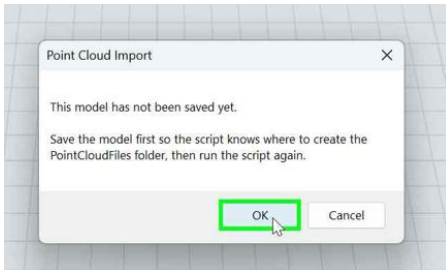
Verify the source coordinate units before importing. The importer scales source coordinates to the model length units; it does not infer source units from the file. The library does not expose the source script settings described in Appendix A.

Z-Up is the recommended source export setting for the supplied test workflow. The documented draw trigger also applies a fixed -90° rotation about X. Check the displayed orientation with a known scan before using other export conventions; do not assume that selecting Z-Up alone guarantees the final orientation.

Importing a Point Cloud

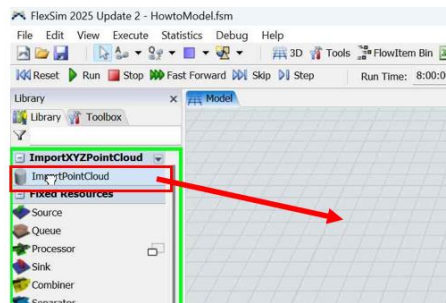
Saving the Model

The FlexSim model must be saved somewhere before running the import tool. The tool manages point cloud files in a folder that will reside in the same directory as the model. If the model has not been saved before running the import tool you will be shown a message and the import will be terminated upon clicking ok.



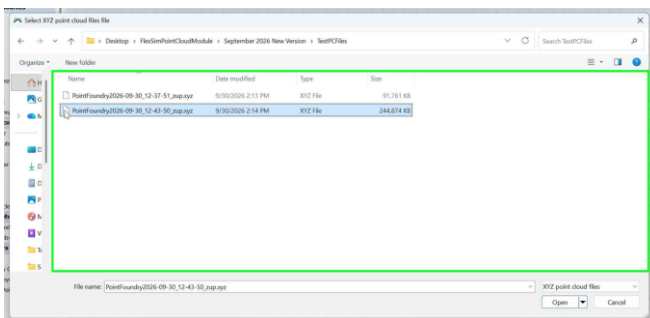
Opening the Importer

Initiate importing by dragging the "ImportPointCloud" object, from the user library "ImportXYZPointCloud", into the Model 3D space. This will open a file selection dialog.



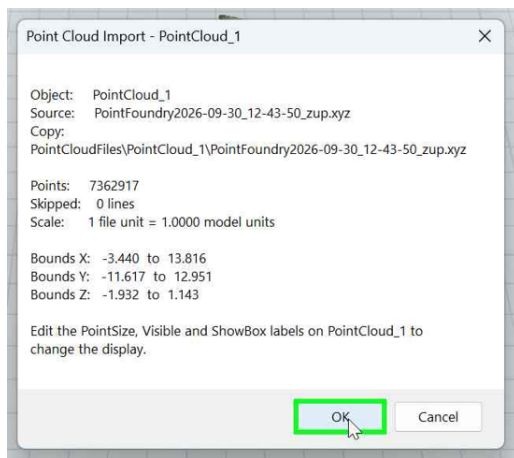
Selecting the XYZ File

Use the file selection dialog to navigate to the point cloud file you would like to import and double click on the file.

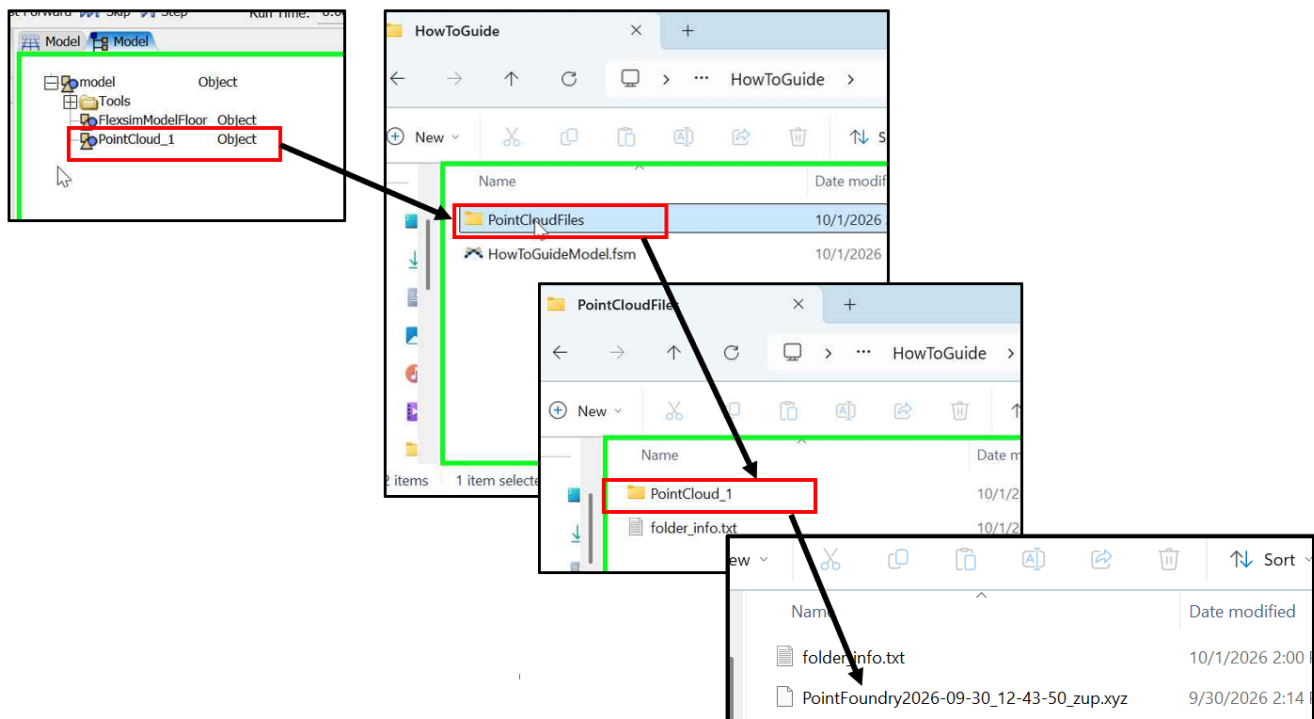


Reviewing the Import Results

When the import procedure completes, a summary window will appear providing details about the point cloud and the import.



*Each import copies the point cloud into a new folder in the same directory as the model. The tool tries to keep naming the same in the model tree, and the file directory

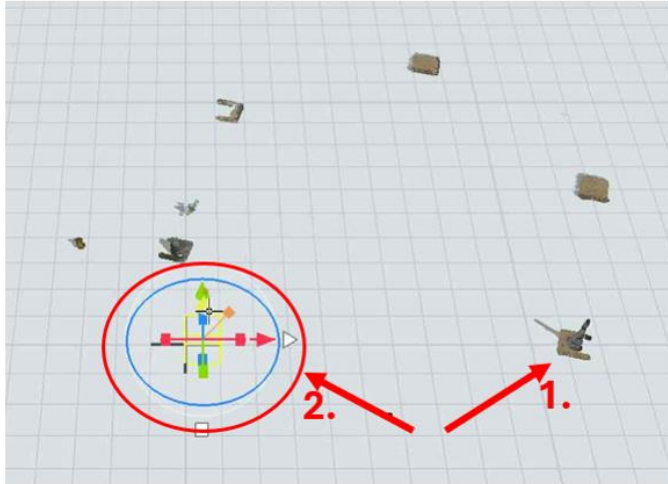


Adjusting the Imported Point Cloud

Locating and Selecting the Cloud

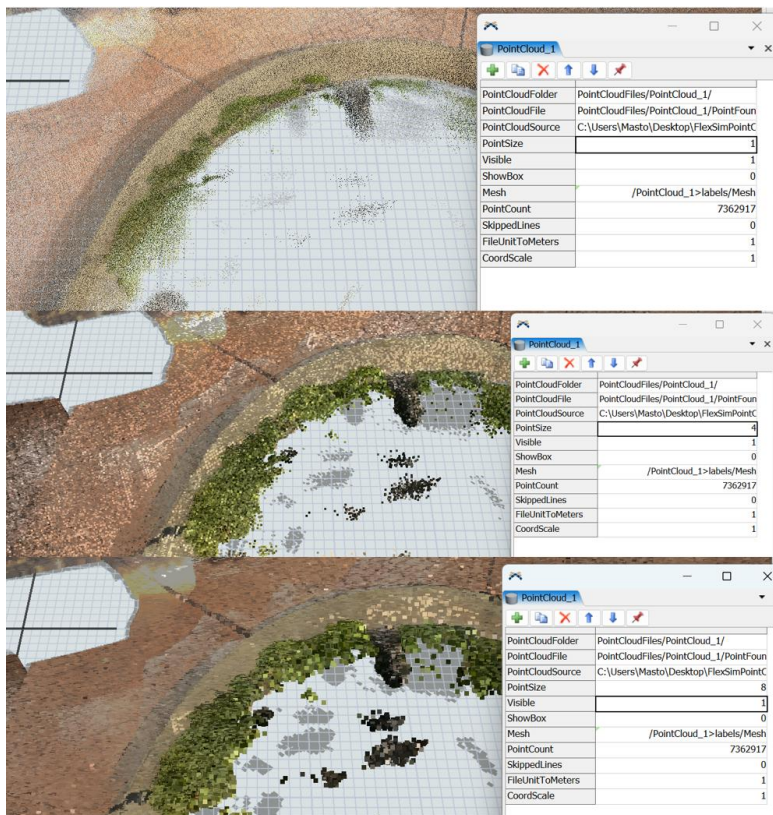
The point cloud should now be visible in the models 3D environment. If it's not visible, you can find it in the model tree with the node name beginning with "PointCloud_".

1. Click on any visible portion of the point cloud.
2. The **3D gumball/Manipulator tool** becomes visible.
3. Use the **3D gumball/Manipulator tool** to move & rotate the point cloud object.



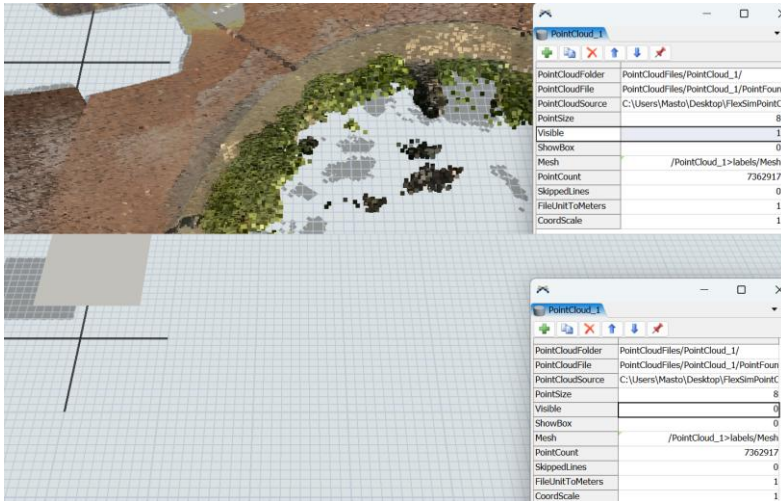
Changing Point Size

Changing the value of the PointSize Label (as illustrated Image1: 1, Image2: 4, Image3: 8).



Controlling Cloud Visibility

Hide the point cloud by setting the label “Visible” value to 0.



Object Containing Point Cloud

A “Plane” shape is what contains the point cloud data. You can make this shape visible by setting the label “ShowBox” to 1. Additionally, the “Plane” becomes visible if the “Visible” label value is set to 0

Display Controls and Object Labels

Edit these three labels on the created Visual Tool without re-importing:

Label	Initial value	Behavior
PointSize	4	Point size in pixels. A missing or nonpositive value falls back to 4.
Visible	1	0 skips drawing the cloud; another value permits cloud drawing.
ShowBox	0	1 permits the Visual Tool's default shape to draw along with the cloud.

When Visible = 0, the trigger returns control to the default Visual Tool drawing code. Its default shape can therefore remain visible even though the cloud is hidden.

The mesh is drawn in model scale so changing the Visual Tool's size does not stretch the cloud. The object initially has location (0, 0, 0) and size (1, 1, 1).

Other labels record the import:

Label	Contents
PointCloudFolder	Relative path to the cloud's subfolder.
PointCloudFile	Relative copied-file path when copying is enabled; original absolute path otherwise.
PointCloudSource	Original absolute source path.
Mesh	Generated point mesh used by the On Draw trigger.

Label	Contents
PointCount	Number of vertices added.
SkippedLines	Number of rejected rows during mesh construction.
FileUnitToMeters	Source unit conversion used during import.
CoordScale	Applied source-to-model coordinate multiplier.

Path labels record file references; the installed On Draw trigger renders Mesh and does not reload the XYZ file. Editing path labels, the XYZ file, or unit labels does not automatically change the mesh.

Saving and Sharing the Model

Point Cloud data is stored in the model and will be visible even if the point cloud files are not shared with the model.

Save functionality operates as normal.

Project Files and Model Portability

A typical import produces the following relative paths:

Path	Purpose
YourModel.fsm	Your saved FlexSim model; the actual model filename may differ.
PointCloudFiles/folder_info.txt	Parent-folder marker.
PointCloudFiles/PointCloud_1/folder_info.txt	Cloud-folder marker containing object and source details.
PointCloudFiles/PointCloud_1/scan.xyz	Source copy, when enabled; the actual source filename is retained.

Move or distribute the model and its PointCloudFiles folder together to preserve relative source references. With copySourceFile = 0, the recorded source reference remains an absolute path to the original file.

The source copy runs asynchronously through `cmd.exe /c copy /Y`. The current import reads the original source directly. A successful mesh build or summary does not verify that the copy has completed or succeeded; check the copied file before moving or distributing the project.

Folder markers and import numbering are described in Appendix B.

Troubleshooting

Symptom	Check or action
Model-not-saved message	Save the model to disk, then run the importer again.
Cannot create or write a folder	Check the saved model folder's permissions and whether the location is writable.
Unsupported file type	Convert or export to XYZ text; changing an extension alone does not convert the contents.
No XYZ data detected	Check the first sample lines, delimiter, header, and column order.
Cloud is too large or too small	Verify source and model units. For the library, prepare the file to match its source-unit preset and import again. For the standalone script, correct fileUnitToMeters.
Cloud is absent from the view	Check Visible, PointCount, point size, and the printed bounds. Large source coordinates may place it far from the model origin.
Orientation is unexpected	Review the source coordinate convention and the fixed X-axis draw rotation. There is no orientation setting at the top of the script.
Colors are wrong	Confirm columns 4–6 are RGB. For the library, prepare colors in the default 0–255 range. For the standalone script, select the appropriate colorScale.
Visual Tool shape remains after hiding the cloud	Visible = 0 allows default object drawing; it hides only the custom cloud rendering.
Copied XYZ is missing	Check the destination file and allow the background copy to finish. The importer does not confirm copy success.
Many lines are skipped	Check for stray text, short rows, inconsistent separators, and leading whitespace in X tokens.
Import or viewport is slow	Reduce the point count in a point-cloud editor before import. The script adds one vertex per accepted row and has no downsampling or level-of-detail controls.

Supported Input Format

The importer accepts .xyz text files containing one point per line. The first three columns must contain the X, Y, and Z coordinates.

For files containing color information, the expected layout is:

X Y Z R G B

When the first accepted sample row contains six or more columns, columns four through six are interpreted as red, green, and blue values. Additional columns after RGB are ignored.

XYZ Coordinates Only

```
0.6983 3.2403 -1.2932
0.7050 3.2450 -1.2800
0.7120 3.2510 -1.2750
```

Points without RGB values are displayed using the configured default gray color.

XYZ Coordinates with RGB Color

```
0.6983 3.2403 -1.2932 84 89 90
0.7050 3.2450 -1.2800 120 135 150
0.7120 3.2510 -1.2750 255 255 255
```

For the library workflow, prepare RGB values in the default range of 0–255. If a conversion produces normalized values in the range 0.0–1.0, convert those color values to 0–255 before using the library. When running the standalone script instead, set `colorScale = 1.0` to accept normalized RGB values; see Appendix A.

Internally, the importer divides each color component by `colorScale`; it does not clamp values. The standalone script exposes this setting.

Files containing four or five columns do not provide complete RGB data and are therefore displayed using the default gray color.

If additional point attributes are present, RGB values must still occupy columns four through six. For example, a file containing intensity before RGB is not compatible with the expected layout because the intensity value would be interpreted as the red component.

Optional Header

```
X,Y,Z,R,G,B
0.6983,3.2403,-1.2932,84,89,90
0.7050,3.2450,-1.2800,120,135,150
```

Use one consistent delimiter throughout the file, including the header. For reliable header handling, place the header on the first physical line without preceding blank lines. The parser detects a header from the first token's initial character, rather than from specific column names.

Parsing Limits

- Only .xyz filenames are accepted, regardless of letter case.
- Delimiter detection prefers comma, then tab, then space. Semicolons and decimal commas are unsupported.
- Use a decimal point for fractional values. Repeated spaces are handled by dropping empty tokens.
- Keep numeric tokens free of leading whitespace, particularly in comma-separated or tab-separated rows; not every mesh-build path trims individual tokens.
- A row with at least three tokens and an X token beginning with a numeric-looking character is converted with `toNum()`. This is a lightweight check, not complete numeric validation of XYZ or RGB.
- Blank lines are ignored. Lines with fewer than three usable tokens or an X token that fails the initial-character check are counted as skipped during mesh construction. Blank lines and the configured header skip are not counted as skipped.
- Multiple header or comment lines are not a supported input contract. Clean the file before importing.

Appendix A Standalone Script Reference

This appendix applies only when running `xyzPointCloudImport.fs` directly. The library workflow is described earlier in this guide. Source settings are not exposed through the library.

Running the Standalone Script

Use a FlexSim script execution context that supports the APIs used in the source.

1. Save your FlexSim model. If the model has never been saved, the script displays a message and stops; it does not open Save As automatically.
2. Open `xyzPointCloudImport.fs` in a text editor and review the settings at the top, especially `fileUnitToMeters`.
3. Paste the complete source into the FlexSim script editor or execution context you use to run standalone FlexScript. The supplied file does not install a menu command or toolbar button.
4. Execute the script once and select your `.xyz` file. The browser initially opens at the model's `PointCloudFiles` folder; browse elsewhere if needed.
5. Review the summary and console output. A successful import creates a Visual Tool such as `PointCloud_1`.
6. Adjust the object's display labels as needed, verify the background source copy has finished if enabled, and save the model again.

Running the script again creates another cloud; it does not update an existing cloud. The script returns the created object on success and returns 0 for its explicit early-stop paths.

Standalone Script Configuration

Edit these values before running the script. Changing the source settings after an import does not rebuild the existing mesh.

Setting	Default	Purpose
<code>folderName</code>	"PointCloudFiles"	Parent folder beside the saved model.
<code>namePrefix</code>	"PointCloud_"	Prefix for new object and subfolder names.
<code>fileFilter</code>	"*.xyz"	File browser filter. The separate extension check still requires <code>.xyz</code> .
<code>fileFilterDesc</code>	"XYZ point cloud files"	File browser filter description.
<code>markerName</code>	"folder_info.txt"	Marker used to recognize folders created by the script.
<code>maxTries</code>	200	Maximum marker-write attempts after folder creation. There is no delay between attempts.
<code>sampleLines</code>	10	Maximum sampled non-empty data candidate lines.
<code>fileUnitToMeters</code>	1.0	Size of one source coordinate unit in meters.
<code>colorScale</code>	255.0	Divisor used to normalize RGB components. Use a positive, nonzero value.

Setting	Default	Purpose
defaultGray	0.7	R, G, and B values for points without usable RGB columns.
pointSize	4	Initial point size in pixels.
copySourceFile	1	1 starts a source-file copy; 0 keeps the original source path.
showSummary	1	1 displays the final popup; console messages are printed either way.
progressEvery	250000	Console progress interval in imported points. Use a positive integer.

Coordinate Units

The script assumes the point cloud units are Meters and reads the model's length-unit conversion and calculates:

```
coordScale = fileUnitToMeters / modelUnitToMeters
mesh coordinate = source coordinate × coordScale
```

Source units	fileUnitToMeters
Meters	1.0
Millimeters	0.001
Centimeters	0.01
Inches	0.0254
Feet	0.3048

For example, if the source uses meters and the model uses inches, a source coordinate of 1.0 becomes approximately 39.3701 model units. Source units are configured manually; the script does not infer them from the file.

Coordinates retain their original origin; the importer does not center or translate them. The generated draw trigger applies a fixed -90° rotation about X. Printed bounds describe the scaled mesh coordinates before that draw rotation, rather than the final displayed world-space bounds.

Appendix B Developer Reference

Folder Recognition and Import Numbering

Folder recognition relies on `folder_info.txt`, because the script uses `fileexists()` for file checks. Keep marker files in place. An existing subfolder without its marker can be treated as available, and `/Y` permits overwriting a same-named destination file.

Numbering starts at the count of top-level model objects whose names begin with `PointCloud_`, plus one. The script increments that number until it finds no exact object-name match and no subfolder marker. It does not necessarily fill numbering gaps or select the highest existing number plus one.

Input Validation

Validation samples up to ten non-empty data candidate lines by default. A bad sampled line is logged, but does not itself reject the file if another sampled row establishes at least three columns. Later malformed values may therefore pass the initial check.

Application Flow

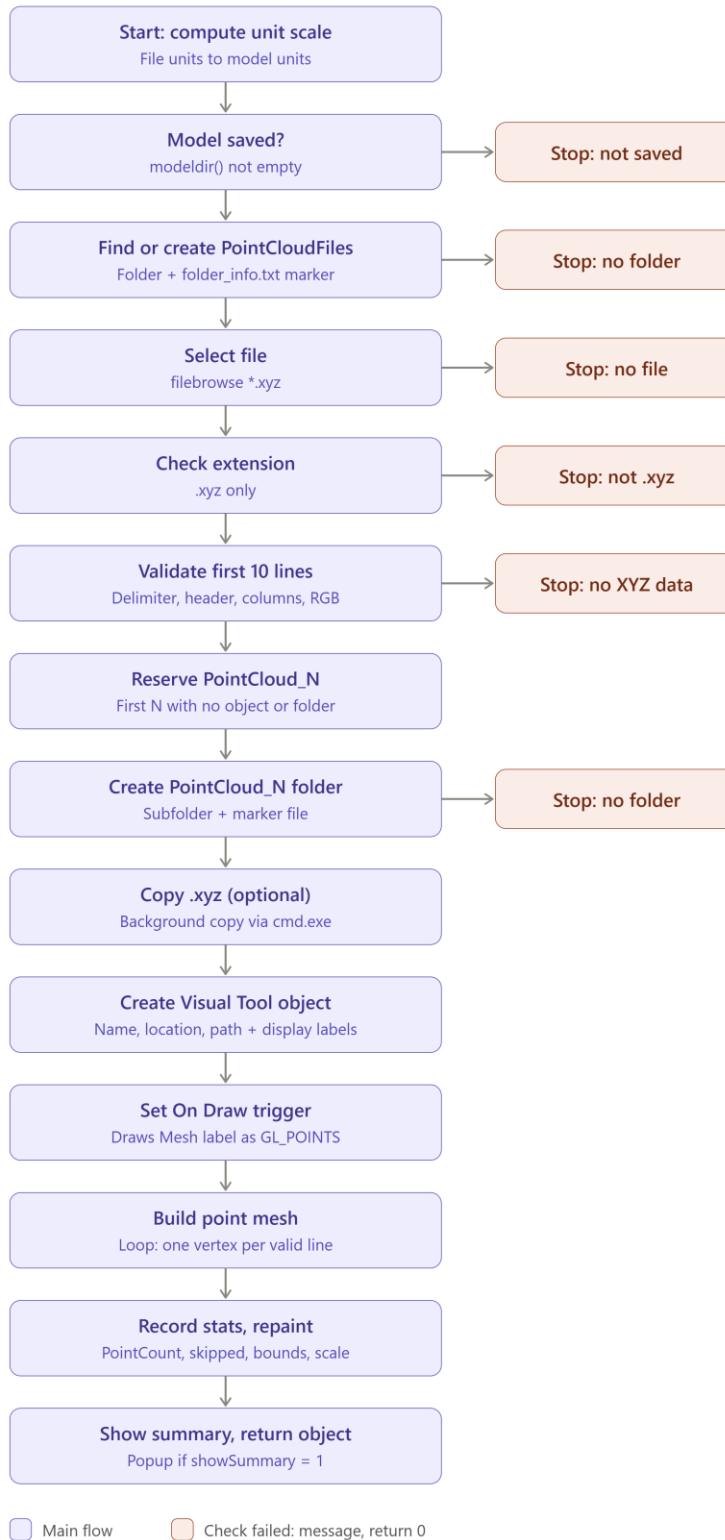
The following sequence describes the importer's internal processing and explicit stop conditions. It is reference material rather than a library operating procedure.

- 1. Check the saved model.** If the model has no saved directory, display a message and stop.
- 2. Prepare the parent folder.** Find or create `PointCloudFiles` and its marker. Stop if the marker cannot be written.
- 3. Select and check the source.** Cancellation stops the import. A missing file or an extension other than `.xyz` displays an error and stops.
- 4. Sample the contents.** Detect the delimiter, optional header, and column count. Stop if no XYZ rows are detected.
- 5. Reserve a name and folder.** Find an unused `PointCloud_N` name, create the subfolder, and write its marker. Stop if the marker cannot be written.
- 6. Optionally start a source copy.** When copying is enabled, start the background copy. Continue using the original source for this import.
- 7. Create the display object.** Create a Visual Tool, attach labels, and install its On Draw trigger.
- 8. Build and report the mesh.** Read the source, add accepted points, store counts, print bounds, repaint, and optionally display the summary.

The parent folder may already have been created when selection is canceled or validation fails. Later failures can leave folders or a partially configured object; there is no rollback routine.

Application Diagrams

Diagram 1:



Implementation and Contribution Notes

The importer is intentionally maintained as one procedural FlexScript, organized into settings and clearly labeled stages. Contributions should preserve this execution structure unless an alternative has been verified in the intended FlexSim script context.

The mesh-build stage makes one complete pass through the original source after the initial sample pass. Each accepted row becomes a vertex with position and ambient/diffuse color attributes. The installed trigger renders the mesh with `GL_POINTS` and reads the display labels every frame.

Current limitations include lightweight input checks, asynchronous copying without completion verification, marker-based folder recognition, a fixed display rotation, and no automatic rollback or reload. Review these behaviors when extending the tool.

Before releasing a changed script, manually verify coordinates-only and RGB files, each supported delimiter, a first-line header, source/model unit conversion, canceled selection, repeated imports, display controls, and project relocation with the copied sources. Compare a small known cloud's point count and bounds with expected values.

License

Copyright (c) 2026 Simulation Bureau LLC.

This project is released under the MIT License. Distribute the full `LICENSE.TXT` with the script and documentation, and retain the required copyright and permission notices in redistributed copies or substantial portions.

FlexSim is a separate proprietary product. This project's license applies to the importer and its documentation, and does not grant a FlexSim license. FlexSim is a trademark of its respective owner. This project does not imply vendor endorsement.